

MLD-ASCON: A Machine Learning Framework for Distinguishing Reduced-Round ASCON Permutations

Machine-learning-assisted distinguishers for studying the distinguishability of reduced-round ASCON permutations through multiple cryptographic feature representations.

Overview

MLD-ASCON investigates whether machine-learning models can distinguish reduced-round ASCON permutation outputs from random differential behavior, and how this distinguishability changes as the number of permutation rounds increases. The framework evaluates multiple representations—including **differential**, **integral**, **cube**, and **intermediate-state** features—using several machine-learning models—including **XGBoost**, **CNN**, **logistic_regression**, **MLP**, **Random Forest**, while incorporating cross-round and cross-word generalization, statistical validation, SHAP-based interpretability, diffusion-threshold analysis, and classical differential comparison.

Experiments across reduced-round ASCON configurations show a clear transition in distinguishability as diffusion increases: machine-learning distinguishers achieve near-perfect performance on several lower-round configurations, while performance approaches random guessing around the diffusion threshold. The project therefore uses ML not only as a classification mechanism, but also as an experimental tool for studying where and how cryptographic structure becomes statistically indistinguishable from random behavior.

Research focus: Reduced-round ASCON · ML Cryptanalysis · Cryptographic Distinguishers · Differential Cryptanalysis · Feature Representations · Explainable ML · Diffusion Analysis · Statistical Validation

Key Features

MLD-ASCON is organized as a modular experimental framework for investigating machine-learning-based distinguishers for reduced-round ASCON permutations. Its capabilities span dataset generation, machine-learning classification, cryptanalytic analysis, statistical validation, interpretability, and reproducible experimentation.

Cryptographic Dataset Generation

The framework provides dedicated generators for multiple cryptographic feature representations:

- **Random-state datasets** — randomly generated ASCON states used as baseline

inputs.

- **Differential datasets** — paired states generated using controlled input differences and evaluated through the ASCON permutation.
- **Integral datasets** — feature representations based on integral-style state behavior.
- **Cube datasets** — cube-oriented feature representations for reduced-round distinguishability analysis.
- **Intermediate-state datasets** — differential state representations extracted from selected intermediate permutation rounds.
- **Configurable reduced-round generation** — datasets can be generated for different ASCON permutation round counts and sample sizes.
- **320-bit state representation** — ASCON's five 64-bit words (x0–x4) are represented as binary features for machine-learning experiments.

Machine-Learning Distinguishers

The framework evaluates multiple machine-learning approaches rather than relying on a single classifier:

- Logistic Regression
- Random Forest
- XGBoost
- Multi-Layer Perceptron (MLP)
- Convolutional Neural Network (CNN)

This allows the experiments to compare linear, ensemble-tree, gradient-boosting, and neural-network approaches under the same distinguishability setting.

Feature-Representation Analysis

MLD-ASCON compares the distinguishability provided by different representations of ASCON state behavior:

- Raw state representation
- Differential representation
- Integral representation
- Cube representation

- Intermediate-state representations
- Word-level analysis across x0, x1, x2, x3, and x4

The framework can therefore evaluate not only whether an ML model distinguishes the permutation, but also which representation exposes the strongest statistical structure.

Cryptanalytic Experiments

The repository contains experiments covering the following aspects:

Experiment	Purpose
accuracy_vs_rounds	Measures ML distinguishability as the number of ASCON rounds increases
differential_propagation	Measures the propagation of differential differences through reduced-round permutations
intermediate_decay	Studies the evolution/decay of distinguishability across intermediate states
feature_importance	Computes permutation-based feature importance for intermediate-state classification
importance_heatmap	Visualizes feature importance across the 320-bit state
feature_comparison	Compares ML performance across feature representations
model_comparison	Compares Logistic Regression, Random Forest, and XGBoost across representations
rounds_comparison	Evaluates distinguishability across increasing ASCON round counts
statistical_validation	Estimates variability and confidence intervals for ML distinguishability
statistical_model_benchmark	Provides statistical comparison of model performance
cross_round_generalization	Tests whether models trained at one round generalize to another
cross_word_generalization	Tests generalization across ASCON state words
differential_patterns	Evaluates multiple differential patterns/configurations

Experiment	Purpose
shap_analysis	Performs SHAP-based bit-level and word-level model interpretation
diffusion_threshold	Relates diffusion growth to the observed ML distinguishability boundary
classical_differential	Establishes a classical differential-cryptanalysis baseline
statistical_classical	Provides statistical validation of the classical baseline
explainability_diffusion	Correlates model explainability with differential diffusion
cross-round / feature generalization analyses	Evaluates robustness of learned distinguishers beyond their training configuration

Explainable Machine Learning

The framework goes beyond classification accuracy by investigating what the models learn:

- **Permutation feature importance** at the bit level.
- Aggregation of bit-level importance into **ASCON word-level importance**.
- **SHAP-based feature attribution** for individual state bits.
- Normalized SHAP importance across the five ASCON state words.
- Visualization of the most influential state bits.
- Correlation between learned feature importance and cryptographic diffusion behavior.

This provides an interpretable view of the state structure exploited by the ML distinguishers.

Diffusion and Distinguishability Analysis

The framework explicitly studies the relationship between ASCON's diffusion behavior and ML performance:

- Average Hamming-distance measurements across rounds.
- Round-wise differential propagation analysis.

- ML accuracy measured alongside diffusion.
- Empirical estimation of a **distinguishability threshold**.
- Analysis of the transition from distinguishable to approximately random behavior.
- Correlation between diffusion measurements and ML accuracy.
- Combined diffusion—explainability analysis.

Statistical Validation

To avoid relying solely on individual accuracy measurements, the framework includes:

- Repeated evaluation of distinguishability.
- Mean performance measurements.
- Standard-deviation estimates.
- Confidence intervals.
- Error-bar visualizations.
- Statistical comparison between ML and classical distinguishers.

This provides an experimental basis for assessing whether observed distinguishability is stable rather than an artifact of a single train/test split.

Classical Cryptanalysis Baseline

MLD-ASCON does not treat machine learning in isolation. It includes a classical differential baseline based on:

- Differential Hamming-weight behavior.
- Round-wise classical distinguishability.
- Comparison between classical and ML accuracy.
- Statistical validation of classical results.

This enables the ML-based approach to be evaluated in the context of conventional cryptanalytic distinguishability rather than as an isolated classification problem.

Generalization Analysis

The framework evaluates whether learned distinguishers remain effective outside their original training configuration:

- **Cross-round generalization** — train at one round and evaluate at another.

- **Cross-word generalization** — train using one ASCON word and evaluate on other words.
- Comparison of generalization behavior across feature representations.
- Analysis of whether learned patterns correspond to broader cryptographic structure or configuration-specific artifacts.

Visualization and Quantitative Outputs

Experiments produce both machine-readable and visual outputs, including:

- CSV datasets and evaluation tables.
- JSON experiment manifests.
- Accuracy-vs-round plots.
- Differential-propagation plots.
- Model-comparison plots.
- Feature-importance plots.
- SHAP visualizations.
- Diffusion-threshold plots.
- Statistical error-bar plots.
- Feature-importance heatmaps.

Reproducible One-Step Experiment Pipeline

The repository includes a top-level replication runner:

```
python run_experiments.py --continue-on-error
```

The pipeline orchestrates dataset generation and experiment execution while maintaining isolated run directories:

```
run_experiments_datasets/  
└─ <RUN_ID>/  
  
run_experiments_result/  
└─ <RUN_ID>/  
    └─ artifacts/  
        └─ logs/  
            └─ run_manifest.json
```

Modular Research Architecture

The project separates the experimental workflow into independent modules:

```
datasets/  
  ↓  
models/  
  ↓  
experiments/  
  ↓  
results/  
  ↓  
replication / audit artifacts
```

This makes individual dataset generators, models, and cryptanalytic experiments independently testable while allowing the complete research pipeline to be reproduced through a single command.

Each run records execution metadata, timestamps, generated datasets, experiment artifacts, standard output/error logs, and pipeline status, providing an auditable path from dataset generation to final experimental results.

Repository Navigation

The repository is organized around the experimental workflow of MLD-ASCON, separating cryptographic primitives, dataset generation, machine-learning models, experiments, results, and reproducibility artifacts.

```
MLD-ASCON/  
|  
+---ascon  
|  |  constants.py  
|  |  diffusion.py  
|  |  permutation.py  
|  |  round.py  
|  |  sbox.py  
|  |  state.py  
|  |  utils.py  
|  |  __init__.py  
|  
|  
+---datasets  
|  |  cube_dataset.py  
|  |  differential_dataset.py  
|  |  integral_dataset.py  
|  |  intermediate_dataset.py  
|  |  random_dataset.py  
|  |  __init__.py  
|  
|  
+---experiments  
|  |  accuracy_vs_rounds.py  
|  |  classical_differential.py
```

```

| | cross_round_generalization.py
| | cross_word_generalization.py
| | differential_patterns.py
| | differential_propagation.py
| | diffusion_threshold.py
| | explainability_diffusion.py
| | feature_comparison.py
| | feature_importance.py
| | importance_heatmap.py
| | intermediate_decay.py
| | model_comparison.py
| | rounds_comparison.py
| | shap_analysis.py
| | statistical_classical.py
| | statistical_model_benchmark.py
| | statistical_validation.py
| | __init__.py
|
|
+---models
| | cnn.py
| | logistic_regression.py
| | mlp.py
| | random_forest.py
| | xgboost_model.py
| | __init__.py
|
|
+---notebooks
+---results
| +---Archived_Datasets
| | stat_cube_r2.csv
| | stat_cube_r3.csv
| | stat_cube_r4.csv
| | stat_cube_r5.csv
| | stat_differential_r2.csv
| | stat_differential_r3.csv
| | stat_differential_r4.csv
| | stat_differential_r5.csv
| | stat_integral_r2.csv
| | stat_integral_r3.csv
| | stat_integral_r4.csv
| | stat_integral_r5.csv
| | tmp_cube_r2.csv
| | tmp_cube_r3.csv
| | tmp_cube_r4.csv
| | tmp_cube_r5.csv
| | tmp_differential_r2.csv
| | tmp_differential_r3.csv
| | tmp_differential_r4.csv
| | tmp_differential_r5.csv
| | tmp_integral_r2.csv
| | tmp_integral_r3.csv
| | tmp_integral_r4.csv
| | tmp_integral_r5.csv
|
|
+---Reproducibility_Datasets

```



```

| | cube_r4.csv
| | differential_r2.csv
| | differential_r3.csv
| | differential_r4.csv
| | differential_r5.csv
| | diff_p1.csv
| | diff_p2.csv
| | diff_p3.csv
| | diff_p4.csv
| | diff_p5.csv
| | integral_r4.csv
| | integral_x0_r4.csv
| | integral_x1_r4.csv
| | integral_x2_r4.csv
| | integral_x3_r4.csv
| | integral_x4_r4.csv
| | intermediate_round1.csv
| | intermediate_round2.csv
| | intermediate_round3.csv
| | intermediate_round4.csv
| | random_vs_ascon_r4.csv
| |
| +---Research_Outputs
| | accuracy_vs_rounds.png
| | classical_differential.csv
| | cross_round_generalization.csv
| | cross_word_generalization.csv
| | differential_patterns.csv
| | differential_propagation.png
| | diffusion_threshold.csv
| | diffusion_threshold.png
| | explainability_diffusion.png
| | explainability_summary.csv
| | feature_comparison.png
| | importance_heatmap.png
| | intermediate_decay.png
| | model_comparison.csv
| | model_comparison.png
| | rounds_comparison.png
| | rounds_comparison_errorbars.png
| | shap_integral_r4.png
| | shap_word_normalized.png
| | statistical_classical.csv
| | statistical_model_benchmark.csv
| | statistical_results.csv
| | word_importance_round1.png
| | word_importance_round2.png
| | word_importance_round3.png
| |
| +---run_experiments_datasets
| |   └─ <RUN_ID>/
| |       └─ # Datasets generated during a reproducible pipeline run
| |
| |
| +---run_experiments_result
| |   └─ <RUN_ID>/
| |       └─ artifacts/

```

```

|   |   | logs/
|   |   | workspace/
|   |   | run_manifest.json
|
|
+---tests
|   |   | test_avalanche.py
|   |   | test_constants.py
|   |   | test_diffusion.py
|   |   | test_permutation.py
|   |   | test_round.py
|   |   | test_sbox.py
|   |   | test_state.py
|   |   | test_trace.py
|   |   | test_utils.py
|
|   | run_experiments.py
|
|   | .gitignore
|   | README.md
|   | requirements-core.txt
|   | requirements.txt
|
|

```

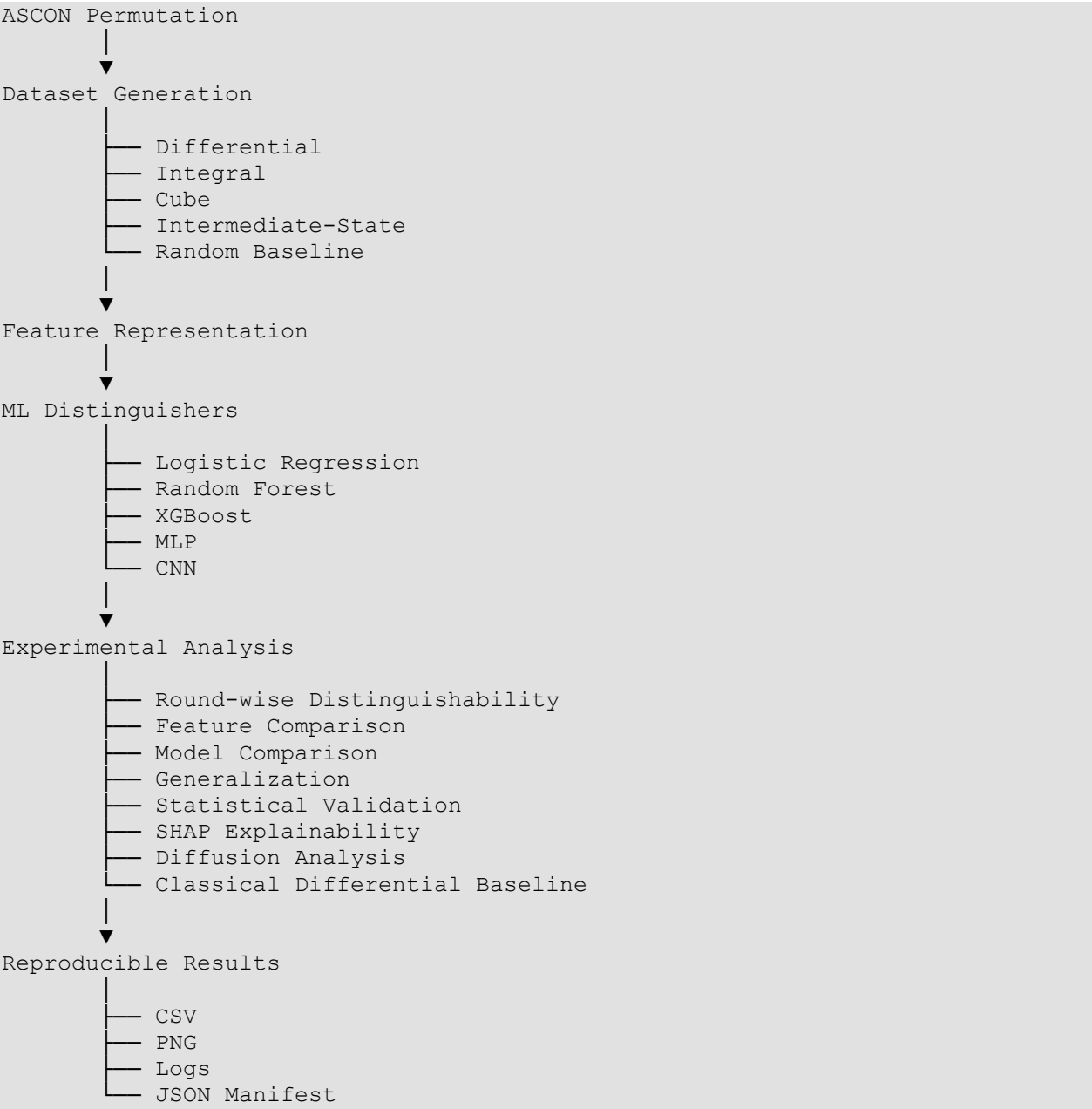
Directory Overview

Directory	Purpose
ascon/	Core ASCON state representation and permutation implementation used by the experiments
datasets/	Generators for random, differential, integral, cube, and intermediate-state datasets
models/	Machine-learning implementations evaluated as cryptographic distinguishers
experiments/	Experimental analyses covering distinguishability, diffusion, generalization, explainability, statistical validation, and classical comparison
results/	Outputs produced during individual experiment execution, including CSV data and visualizations
run_experiments_datasets/	Isolated datasets generated by the one-step reproducibility pipeline
run_experiments_result/	Complete reproducibility artifacts for each pipeline execution, including logs, plots, CSV outputs, workspace data, and the run manifest

Directory	Purpose
run_experiments.py	Top-level orchestration script for executing the complete experimental pipeline

Core Workflow

The repository follows the following research workflow:



Reproducibility Outputs

Each execution of the complete pipeline receives a unique run identifier:

```
run_experiments_result/<RUN_ID>/
```

The corresponding generated datasets are stored separately under:

```
run_experiments_datasets/<RUN_ID>/
```

This separation ensures that generated experimental data and experimental results remain independently identifiable, while the run manifest and execution logs provide an audit trail for the complete experiment.

Installation & Environment Setup

MLD-ASCON is implemented in Python and is designed to run as a local experimental research framework. The experiments operate on datasets generated directly by the repository and do not require downloading an external cryptographic dataset.

Prerequisites

Before installing the project, ensure the following are available:

- **Python:** 3.12 or compatible Python 3.x environment - py 3.12x or below is required for tensorflow
- **Operating System:** Windows, Linux, or macOS
- **Git:** Required for cloning the repository
- **pip:** Python package manager
- **Virtual environment:** Recommended for dependency isolation

GPU / CUDA: The current experimental pipeline does not require CUDA or a dedicated GPU. The machine-learning experiments are designed to run using the available CPU environment.

1. Clone the Repository

```
git clone https://github.com/SaurabhJha19/MLD-ASCON-Machine-Learning-Framework-for-Distinguishing-Reduced-Round-ASCON-Permutations.git
```

2. Create a Virtual Environment

Creating an isolated Python environment is recommended to prevent dependency conflicts with other projects.

Windows

```
python -m venv .venv  
.venv\Scripts\activate.ps1
```

Linux / macOS

```
python3 -m venv .venv  
source .venv/bin/activate
```

After activation, the terminal should indicate that the **.venv** environment is active.

3. Install Dependencies

Install the project's Python dependencies using:

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

4. Verify the Environment

Verify that Python is available in the active environment:

```
python --version
```

A compatible environment should report Python 3.x, with the primary development environment using Python 3.12.

The project can then be checked for basic import/syntax issues with:

```
python -m py_compile run_experiments.py
```

5. Generate Experimental Data

MLD-ASCON does not depend on a separately downloaded benchmark dataset. The experimental datasets are generated locally using the dataset-generation modules.

Individual generators can be executed through Python's module interface:

```
python -m datasets.random_dataset  
python -m datasets.differential_dataset  
python -m datasets.integral_dataset  
python -m datasets.cube_dataset  
python -m datasets.intermediate_dataset
```

These modules generate the data required by the corresponding experiments.

7. Running Individual Experiments

Individual experiments can also be executed independently when reproducing a specific analysis.

```
python -m experiments.accuracy_vs_rounds
python -m experiments.differential_propagation
python -m experiments.intermediate_decay
python -m experiments.feature_importance
python -m experiments.importance_heatmap
python -m experiments.feature_comparison
python -m experiments.model_comparison
python -m experiments.rounds_comparison
python -m experiments.statistical_validation
python -m experiments.statistical_model_benchmark
python -m experiments.cross_round_generalization
python -m experiments.cross_word_generalization
python -m experiments.differential_patterns
python -m experiments.shap_analysis
python -m experiments.diffusion_threshold
python -m experiments.classical_differential
python -m experiments.statistical_classical
python -m experiments.explainability_diffusion
```

Individual experiment scripts generally write their generated CSV data and visualizations to the project's **results/** directory.

7. Run the Complete Experimental Pipeline

For a complete end-to-end replication run, use the top-level experiment runner:

```
python run_experiments.py --continue-on-error
```

The pipeline automatically orchestrates the dataset-generation and experimental stages and records the execution under a unique run identifier.

Generated datasets are stored separately from experiment results:

```
run_experiments_datasets/
└─ <RUN_ID>/
```

while experiment outputs and execution metadata are stored under:

```
run_experiments_result/
└─ <RUN_ID>/
   ├── artifacts/
   ├── logs/
   ├── workspace/
   └─ run_manifest.json
```

8. No External Dataset Download Required

The current MLD-ASCON experimental framework generates its required cryptanalytic datasets locally from the implemented ASCON permutation.

The dataset-generation pipeline covers:

```
Random-state samples
```

```
Differential samples
Integral samples
Cube samples
Intermediate-state samples
```

Consequently, there is currently no separate **download_data.sh** or external dataset-fetching step required for reproducing the core experiments.

Environment Summary

Component	Requirement
Python	Development environment: Python 3.12
Package manager	pip
Environment	Python virtual environment recommended
GPU	Not required
CUDA	Not required by the current pipeline
External datasets	Not required
Dataset generation	Performed locally
Reproducibility	run_experiments.py

Experimental outputs include:

Type	Description
.csv	datasets, metrics, statistical results, and experiment summaries
.png	generated plots and visualizations
.json	run metadata and execution manifest
.txt	module standard-output and error logs

Step-by-Step Guide: Running the Project

This section provides a complete walkthrough for reproducing the MLD-ASCON experimental workflow from a fresh repository checkout.

Step 1 — Clone the Repository

Clone the repository and move into the project directory:

```
git clone https://github.com/SaurabhJha19/MLD-ASCON-Machine-Learning-Framework-for-Distinguishing-Reduced-Round-ASCON-Permutations.git
```

Step 2 — Create a Python Virtual Environment

Creating an isolated environment is recommended to avoid dependency conflicts.

Windows

```
python -m venv .venv  
.venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv .venv  
source .venv/bin/activate
```

After activation, the terminal should indicate that the virtual environment is active.

Step 3 — Install Dependencies

Upgrade pip and install the project's dependencies:

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

The repository uses Python-based scientific computing and machine-learning libraries for dataset processing, model training, evaluation, visualization, and statistical analysis.

Verify the Python installation:

```
python --version
```

The primary development environment for this project is **Python 3.12**.

Step 4 — Verify the Project Structure

Before running the experiments, verify that the main project directories are present:

```
ascon/  
datasets/  
models/  
experiments/  
results/  
run_experiments.py  
requirements.txt
```

The `ascon/` directory contains the permutation implementation, `datasets/` contains dataset generators, `models/` contains the ML distinguishers, and `experiments/` contains the research experiments.

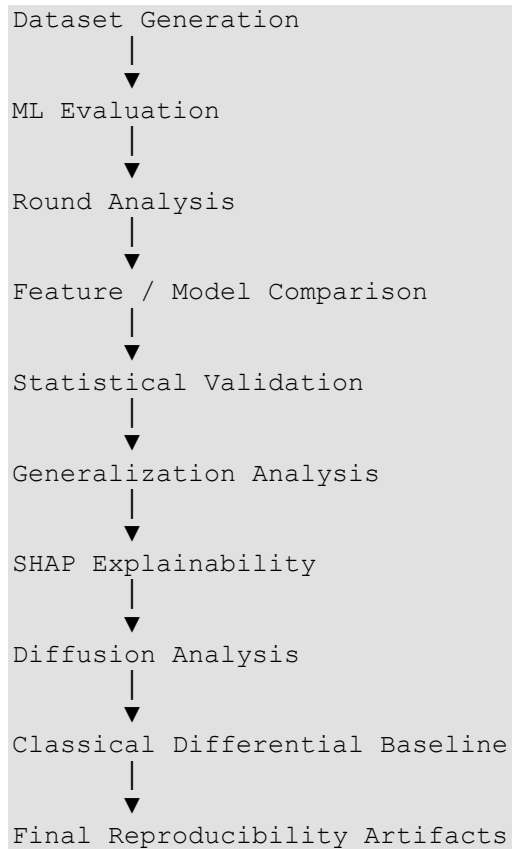
Step 5 — Run the Complete Experimental Pipeline

The recommended method is:


```
python run_experiments.py --continue-on-error
```

The pipeline executes the configured dataset-generation and experimental stages sequentially.

A typical execution looks conceptually like:



The terminal reports the progress of each stage:

```
[1/23] Running datasets.random_dataset ...  
    ✓ Success  
  
[2/23] Running datasets.differential_dataset ...  
    ✓ Success  
  
...  
  
[23/23] Running ...  
    ✓ Success
```

The exact execution time depends on the machine and dataset sizes.

Fallback Execution Procedure: In the event of a top-level orchestrator or runner

failure `run_experiments.py`, please proceed by executing individual experiment scripts independently.

Identify the Run ID

Every complete pipeline execution receives a unique run identifier.

For example:

```
20260816_155812
```

The run identifier is used to associate all generated datasets, logs, experiment outputs, and metadata with one specific execution.

Locate the Generated Datasets

Generated datasets are stored separately from experiment results:

```
run_experiments_datasets/  
└─ <RUN_ID>/
```

For example:

```
run_experiments_datasets/  
└─ 20260816_155812/  
    ├── datasets_random_dataset/  
    ├── datasets_differential_dataset/  
    ├── datasets_integral_dataset/  
    ├── datasets_cube_dataset/  
    └─ datasets_intermediate_dataset/
```

This separation is intentional.

It ensures that the raw/generated experimental data can be identified independently from the plots, metrics, logs, and other experiment artifacts.

Locate the Experimental Results

All outputs generated by the complete pipeline are stored separately under:

```
run_experiments_result/<RUN_ID>/
```

The directory contains:

```
run_experiments_result/  
└─ <RUN_ID>/  
    ├── artifacts/  
    ├── logs/  
    ├── workspace/  
    │   └─ results/  
    └─ run_manifest.json  
artifacts/
```

Contains collected experiment outputs such as:

```
PNG visualizations
CSV evaluation results
statistical summaries
feature-importance outputs
SHAP results
diffusion-analysis outputs
logs/
```

Contains execution logs for individual modules.

These include:

```
stdout
stderr
```

The logs are particularly useful when diagnosing a failed experiment.

```
workspace/
```

Contains the isolated working environment used by the pipeline during execution, including generated experiment results.

```
run_manifest.json
```

Contains run-level metadata and execution information.

This provides an audit trail linking the pipeline execution to its generated outputs.

Inspect the Run Manifest

The run manifest can be opened using any text editor.

```
Get-Content .\run_experiments_result\<RUN_ID>\run_manifest.json
```

The manifest records information about the pipeline execution, including the run identity, execution metadata, and stage results.

The manifest is intended to make individual experimental runs independently identifiable and auditable.

Inspect Experiment Logs

If an experiment fails, inspect its corresponding log under:

```
run_experiments_result/<RUN_ID>/logs/
```

The stderr log contains the Python traceback and error information produced by the failed experiment.

The corresponding stdout log can be inspected with:

```
Get-Content .\run_experiments_result\<RUN_ID>\logs\experiments_<MODULE>_stdout.txt
```

This makes pipeline failures reproducible and easier to diagnose.

Inspect the Generated Results

After a successful run, navigate to:

```
run_experiments_result/<RUN_ID>/workspace/results/
```

The directory contains the CSV and PNG outputs generated by the individual experiments.

Typical outputs include:

```
accuracy_vs_rounds.png
differential_propagation.png
intermediate_decay.png
model_comparison.csv
model_comparison.png
rounds_comparison.png
statistical_results.csv
shap_word_normalized.png
diffusion_threshold.csv
diffusion_threshold.png
classical_differential.csv
statistical_classical.csv
...
```

The exact files depend on the experiment configuration and pipeline version.

Reproduce Individual Experiments

Step 6 — Generate Datasets

The project can generate its experimental datasets locally.

The available dataset generators include:

```
python -m datasets.random_dataset
python -m datasets.differential_dataset
python -m datasets.integral_dataset
python -m datasets.cube_dataset
python -m datasets.intermediate_dataset
```

These correspond to:

Dataset	Purpose
Random	Random baseline/reference samples

Dataset	Purpose
Differential	Controlled differential-state experiments
Integral	Integral-style feature representation
Cube	Cube-based feature representation
Intermediate	Intermediate-state analysis

For normal reproduction, manual execution of these commands is not required because the top-level pipeline invokes the required dataset-generation stages automatically.

Step 7 — Reproduce Individual Experiments

Individual experiments can be executed independently by running :

```
python -m experiments.accuracy_vs_rounds
python -m experiments.differential_propagation
python -m experiments.intermediate_decay
python -m experiments.feature_importance
python -m experiments.importance_heatmap
python -m experiments.feature_comparison
python -m experiments.model_comparison
python -m experiments.rounds_comparison
python -m experiments.statistical_validation
python -m experiments.statistical_model_benchmark
python -m experiments.cross_round_generalization
python -m experiments.cross_word_generalization
python -m experiments.differential_patterns
python -m experiments.shap_analysis
python -m experiments.diffusion_threshold
python -m experiments.classical_differential
python -m experiments.statistical_classical
python -m experiments.explainability_diffusion
```

Experiment Catalogue

Experiment	Purpose
accuracy_vs_rounds	Measures ML distinguisher accuracy as the number of ASCON permutation rounds increases.
differential_propagation	Analyzes how controlled input differences propagate through the ASCON permutation across rounds.
intermediate_decay	Tracks the decay of distinguishability through intermediate ASCON permutation states.
feature_importance	Identifies the input features that contribute

Experiment	Purpose
	most strongly to the ML distinguisher's predictions.
importance_heatmap	Visualizes bit-level feature importance across the 320-bit ASCON state.
feature_comparison	Compares the distinguishability performance of different cryptographic feature representations.
model_comparison	Compares Logistic Regression, Random Forest, and XGBoost as ML distinguishers across feature representations.
rounds_comparison	Compares distinguishability across different ASCON round configurations and feature representations.
statistical_validation	Quantifies the variability and confidence of round-wise ML distinguishability results.
statistical_model_benchmark	Provides statistical benchmarking of the evaluated ML models and their distinguishability performance.
cross_round_generalization	Evaluates whether a distinguisher trained on one ASCON round configuration generalizes to another.
cross_word_generalization	Evaluates whether learned distinguishing patterns generalize across different ASCON state words.
differential_patterns	Evaluates distinguishability under different configurable differential input patterns.
shap_analysis	Uses SHAP to provide bit-level and ASCON word-level explanations of ML distinguisher predictions.
diffusion_threshold	Estimates the empirical round at which increasing diffusion coincides with the loss of ML distinguishability.
classical_differential	Establishes a classical differential distinguisher baseline for comparison with the ML approach.
statistical_classical	Provides statistical validation of the classical differential distinguisher results.
explainability_diffusion	Examines the relationship between SHAP-based feature importance, diffusion, and

Experiment	Purpose
	ML distinguishability.

Step 8 — Re-run After a Failure

If the complete pipeline stops because an experiment fails, inspect the relevant stderr log first.

```
Get-Content .\run_experiments_result\<RUN_ID>\logs\experiments_<MODULE>_stderr.txt
```

After resolving the underlying environment or dependency problem, run:

```
python run_experiments.py --continue-on-error
```

A new run receives a new <RUN_ID>, preserving the previous run for auditability.

Diagnostic Pipeline Execution

For debugging purposes, the pipeline can be instructed to continue after an experiment failure:

```
python run_experiments.py --continue-on-error
```

This mode is useful when the objective is to identify multiple failing modules in a single execution.

For formal reproduction of the research results, a fully successful pipeline execution is preferred.

The complete workflow is:

Using `run_experiments.py`:

```
1. Clone repository
   ↓
2. Create virtual environment
   ↓
3. Install requirements.txt
   ↓
4. Verify Python installation
   ↓
5. Run run_experiments.py
   ↓
6. Record the generated RUN_ID
   ↓
7. Inspect run_manifest.json
```

```
↓
8. Inspect logs/
↓
9. Inspect artifacts/
↓
10. Inspect workspace/results/
↓
11. Review CSV metrics
↓
12. Review generated plots
↓
13. Use individual experiment modules for deeper analysis
```

For Reproducing Individual Experiments

```
1. Clone repository
↓
2. Create virtual environment
↓
3. Install requirements.txt
↓
4. Verify Python installation
↓
5. Run Each Dataset Generation Scripts
↓
6. Run Individual Experiment Scripts
↓
7. Inspect results/
↓
8. Review CSV metrics
↓
9. Review generated plots
```

Quantitative Benchmark Results

MLD-ASCON evaluates reduced-round ASCON distinguishability using multiple feature representations, machine-learning models, and a classical differential baseline. The primary benchmark is classification performance against random/reference samples, complemented by statistical confidence intervals, diffusion measurements, and round-wise analysis.

Round-Wise ML Distinguishability

The following results summarize the observed XGBoost distinguishability across the reduced-round experiments:

Feature Representation	Round 2	Round 3	Round 4	Round 5
Differential	1.0000	0.9998	0.5035	0.5067
Integral	1.0000	1.0000	1.0000	0.4945

Feature Representation	Round 2	Round 3	Round 4	Round 5
Cube	1.0000	1.0000	1.0000	0.5066

Accuracy values close to 1.0 indicate strong distinguishability in the evaluated reduced-round configuration, whereas values close to 0.5 indicate performance approaching random guessing.

The results show a pronounced degradation in distinguishability as additional permutation rounds increase, particularly for the differential representation. This transition is further investigated through diffusion and threshold analysis.

Statistical Validation

The statistical validation experiments provide mean accuracy, variability, and confidence intervals for the evaluated round configurations.

Representation	Round	Mean Accuracy	Standard Deviation	95% CI Half-Width
Differential	2	1.0000	0.0000	0.0000
Differential	3	0.9998	0.0004	0.0003
Differential	4	0.5035	0.0120	0.0075
Differential	5	0.5067	0.0182	0.0113
Integral	2	1.0000	0.0000	0.0000
Integral	3	1.0000	0.0000	0.0000
Integral	4	1.0000	0.0000	0.0000
Integral	5	0.4945	0.0053	0.0033
Cube	2	1.0000	0.0000	0.0000
Cube	3	1.0000	0.0000	0.0000
Cube	4	1.0000	0.0000	0.0000
Cube	5	0.5066	0.0172	0.0107

Statistical reporting note: The confidence intervals reported here correspond to the statistical validation procedure implemented in the repository. They should not be interpreted as five independent full-pipeline repetitions. Independent multi-run confidence estimates can be added as a future extension of the reproducibility pipeline.

Machine-Learning Model Comparison

MLD-ASCON evaluates multiple classifier families under the same distinguishability framework.

Dataset Representation /	Logistic Regression	Random Forest	XGBoost
Differential	0.5050	0.5125	0.5355
Integral	1.0000	1.0000	1.0000
Cube	1.0000	1.0000	1.0000
Intermediate R3	0.99925	1.0000	1.0000

The comparison demonstrates that model performance depends strongly on the underlying feature representation. In particular, the differential representation at the evaluated configuration provides substantially weaker distinguishability than the integral, cube, and selected intermediate-state representations.

MLD-ASCON evaluates multiple classifier families as potential cryptographic distinguishers. The following table reports the benchmark results currently available for the evaluated configurations.

Dataset / Representation	MLP	CNN
Differential	0.5250	0.4990

For the evaluated four-round differential configuration, the corresponding ROC-AUC values were:

Model	Accuracy	ROC-AUC
Logistic Regression	0.5050	0.5122
Random Forest	0.5125	0.5006
XGBoost	0.5355	0.5368
MLP	0.5250	0.5257
CNN	0.4990	0.5031

The results demonstrate that model performance is strongly dependent on the cryptographic representation and round configuration. In the evaluated four-round differential setting, all models remain close to random-guessing performance, with XGBoost providing the highest observed accuracy among the compared models.

Classical Differential Baseline

The ML results are additionally compared with a classical differential baseline:

Round	Classical Accuracy	95% CI Half-Width
2	1.0000	0.0000
3	0.8292	0.0040
4	0.4968	0.0046
5	0.4980	0.0030

The classical baseline exhibits a similar transition toward random-guessing behavior at higher round counts. This provides a reference point for interpreting the additional distinguishability obtained through machine-learning-based analysis.

Diffusion Measurements

The framework also measures differential propagation using average Hamming distance:

Round	Average Hamming Distance
1	7.51 bits
2	49.25 bits
3	140.62 bits
4	159.98 bits

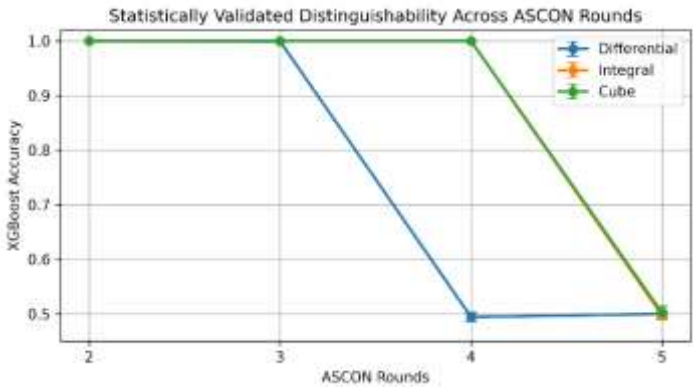
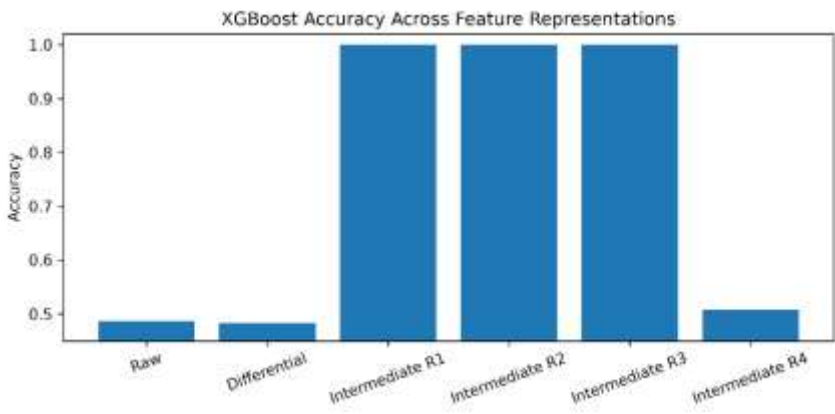
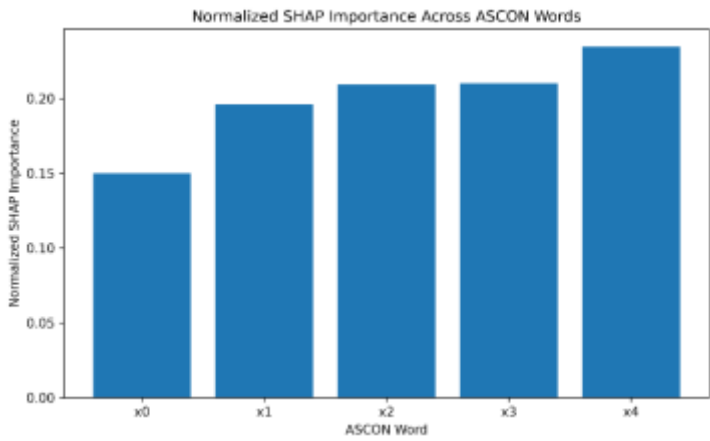
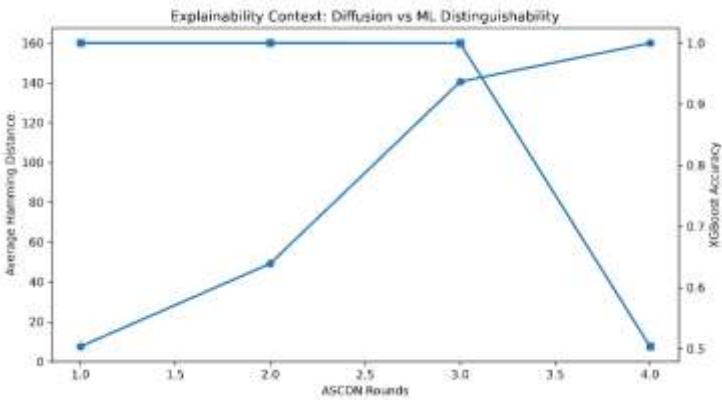
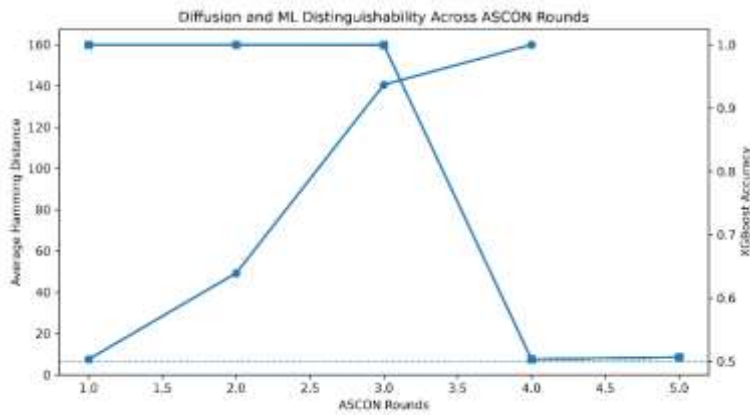
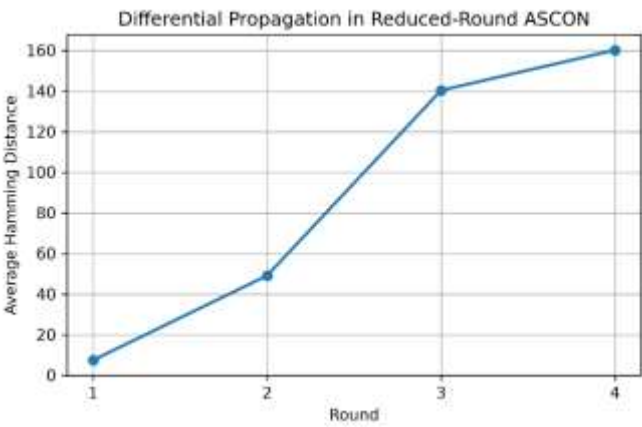
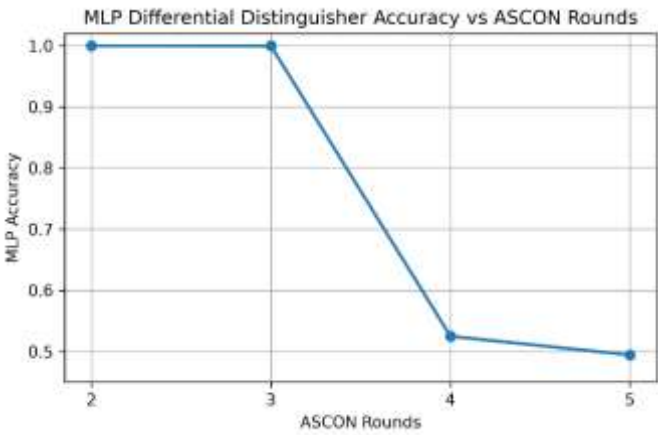
The observed increase in Hamming distance provides an empirical measure of diffusion across the reduced-round permutation. When combined with ML accuracy, these measurements allow the project to examine the relationship between increasing diffusion and decreasing distinguishability.

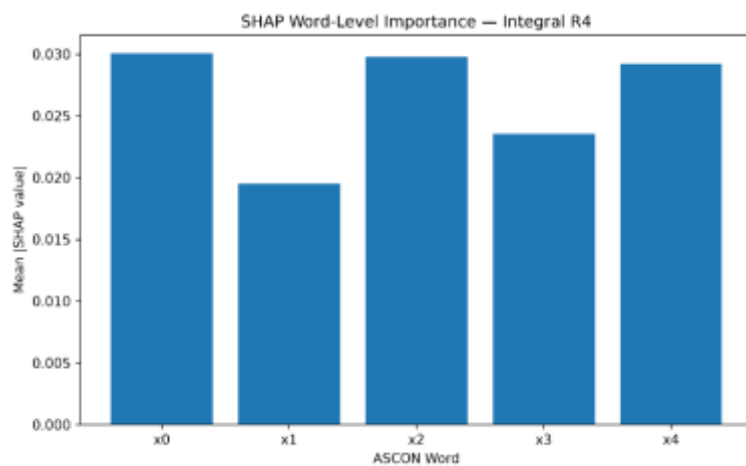
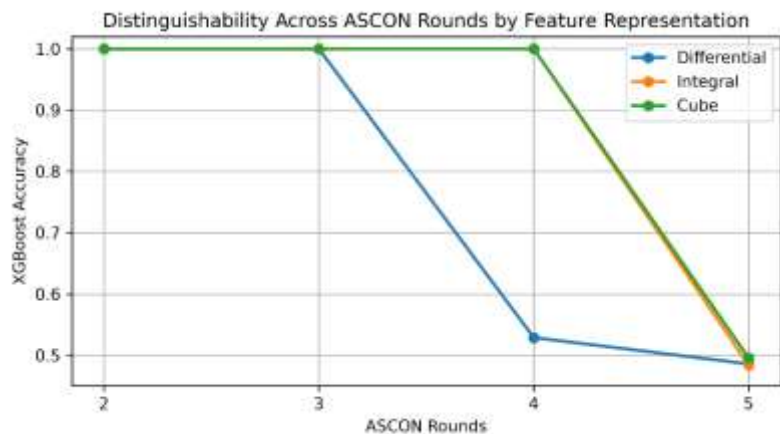
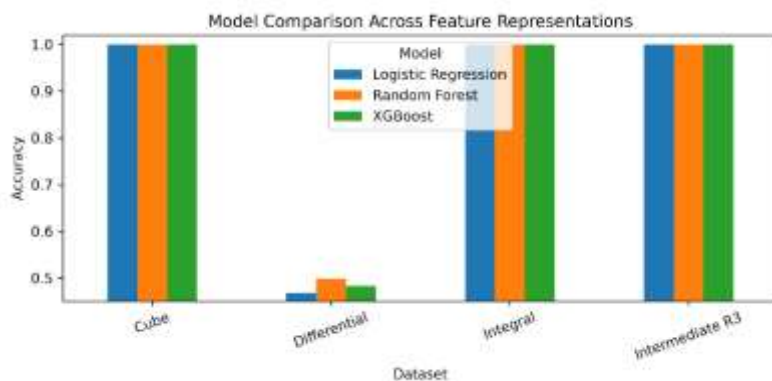
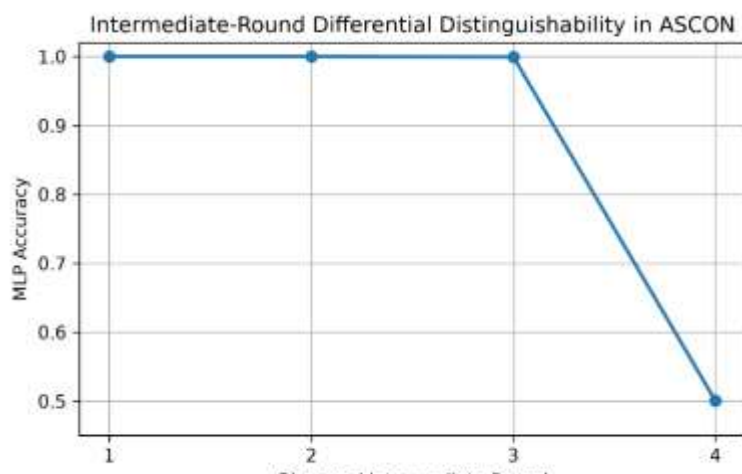
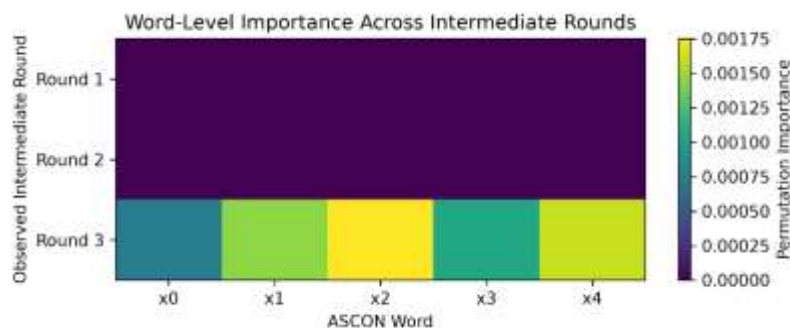
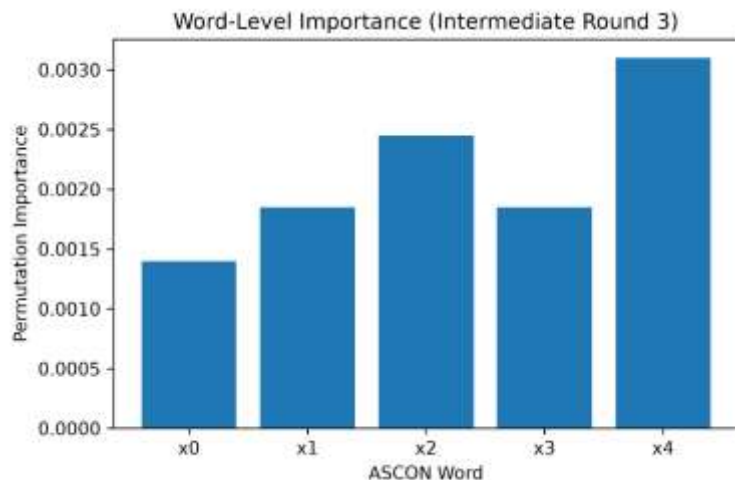
Empirical Distinguishability Transition

The diffusion-threshold analysis identifies Round 4 as the empirical transition point in the evaluated configuration, where the ML distinguishability observed in the analyzed differential setting falls toward random-guessing performance.

This should be interpreted as an empirical threshold for the evaluated experimental setup, rather than as a universal cryptographic property of ASCON.

Visual Results





CSV Results

dataset	classical_accuracy	class0_mean_hw	class1_mean_hw	threshold
results/differential_r2.csv	1	159.9844	49.3924	104.6884
results/differential_r3.csv	0.8317	159.957	140.3386	150.1478
results/differential_r4.csv	0.5054	159.807	160.0142	159.9106
results/differential_r5.csv	0.4982	159.819	159.8754	159.8472

Feature	Train Round	Test Round	Accuracy
Differential	2	3	0.5
Differential	3	4	0.4998
Differential	4	5	0.4938
Integral	3	4	1
Integral	4	5	0.5
Cube	3	4	1
Cube	4	5	0.5

Test Word	Accuracy	Pattern	Accuracy
x0	1	P1 (x0,b0)	0.487
x1	1	P2 (x0,b15)	0.506
x2	1	P3 (x1,b0)	0.497
x3	1	P4 (x2,b31)	0.4985
x4	1	P5 (x4,b63)	0.5035

round	hamming_distance	ml_accuracy	distinguishable
1	7.51	1	TRUE
2	49.25	1	TRUE
3	140.62	0.9998	TRUE
4	159.98	0.5035	FALSE
5		0.5067	FALSE

word	shap_importance	normalized_shap
x0	0.01986883	0.150123408
x1	0.0259212	0.195853449
x2	0.02770827	0.209356057
x3	0.02779566	0.210016352
x4	0.03105602	0.234650734

Dataset	Model	Accuracy	ROC-AUC
Differential	Logistic Regression	0.4675	0.477958
Differential	Random Forest	0.499	0.497377
Differential	XGBoost	0.483	0.478426
Integral	Logistic Regression	1	1
Integral	Random Forest	1	1
Integral	XGBoost	1	1
Cube	Logistic Regression	1	1
Cube	Random Forest	1	1
Cube	XGBoost	1	1
Intermediate R3	Logistic Regression	0.99975	1
Intermediate R3	Random Forest	1	1
Intermediate R3	XGBoost	1	1

round	mean_accuracy	std	ci_95
2	1	0	0
3	0.8306	0.009158481	0.005676485
4	0.50135	0.015765733	0.009771702
5	0.4985	0.00518545	0.003213975

Feature	Rounds	Mean Accuracy	Std	95% CI
Differential	2	1	0	0
Differential	3	0.9996	0.000699206	0.000433372
Differential	4	0.4945	0.013209845	0.008187547
Differential	5	0.5002	0.010163114	0.006299163
Integral	2	1	0	0
Integral	3	1	0	0
Integral	4	1	0	0
Integral	5	0.4972	0.011272385	0.006986697
Cube	2	1	0	0
Cube	3	1	0	0
Cube	4	1	0	0
Cube	5	0.5021	0.018531954	0.011486224

Dataset	Model	Mean Accuracy	Std
Differential R4	Logistic Regression	0.4889	0.012231108
Differential R4	Random Forest	0.503	0.01344123
Differential R4	XGBoost	0.4968	0.007364328
Differential R4	MLP	0.49685	0.00938098
Differential R4	CNN	0.49885	0.008705203
Differential R5	Logistic Regression	0.5022	0.01417196
Differential R5	Random Forest	0.4924	0.015924823
Differential R5	XGBoost	0.5057	0.014967001
Differential R5	MLP	0.4986	0.016734529
Differential R5	CNN	0.5108	0.014680297
Integral R5	Logistic Regression	0.4957	0.011981931
Integral R5	Random Forest	0.4957	0.010520351
Integral R5	XGBoost	0.5007	0.013507611
Integral R5	MLP	0.4935	0.011047876
Integral R5	CNN	0.496	0.014094916
Cube R5	Logistic Regression	0.4913	0.009695933
Cube R5	Random Forest	0.4865	0.014698828
Cube R5	XGBoost	0.4974	0.018679757
Cube R5	MLP	0.4936	0.015005184
Cube R5	CNN	0.4961	0.01889415
Intermediate R4	Logistic Regression	0.502775	0.007843477
Intermediate R4	Random Forest	0.50035	0.006830487
Intermediate R4	XGBoost	0.5017	0.009811982
Intermediate R4	MLP	0.498925	0.006477579
Intermediate R4	CNN	0.5035	0.005288877

Interpretation

Taken together, the benchmark results demonstrate three experimentally observable behaviors:

1. **Strong reduced-round distinguishability** is observed for several feature representations at lower round counts.
2. **Distinguishability decreases substantially as diffusion increases**, with several configurations approaching random-guessing performance at higher rounds.
3. **The choice of feature representation strongly influences ML performance**, indicating that the effectiveness of an ML distinguisher depends not only on the classifier but also on how cryptographic structure is represented.

These results form the quantitative basis for the subsequent explainability, diffusion, generalization, and classical-cryptanalysis analyses.

Conclusion

MLD-ASCON provides a reproducible experimental framework for investigating machine-learning-based distinguishers on reduced-round ASCON permutations. By combining multiple cryptographic feature representations with classical and machine-learning models, statistical validation, generalization testing, SHAP-based explainability, and diffusion analysis, the framework provides both quantitative distinguishability measurements and an interpretable view of the learned cryptographic structure.

The experiments demonstrate that distinguishability is strongly dependent on the permutation round count and feature representation, with several configurations exhibiting a transition toward random-guessing behavior as diffusion increases. The framework is intended as a research and experimentation platform rather than a practical attack against full-round ASCON, and provides a foundation for further investigation into ML-assisted cryptanalysis.